

Informe tecnico de la base de conocimiento vectorial

Etapa 1 | Base de entrega: 012e50b | Auditoria: 12 de agosto de 2026

213,241	1,762	768	50
chunks indexados	documentos cubiertos	dimensiones	consultas validadas

1. Resumen ejecutivo

Cooper implementa recuperacion hibrida multilingue y no generativa sobre el corpus oficial. La ruta entregada combina un encoder publico de Hugging Face, embeddings L2, busqueda exacta FAISS por producto interno, metadata JSONL alineada y evidencia de un grafo GraphML trazable. No intervienen LLM, decoders, BM25, ChromaDB, query expansion, cross-encoders ni reranking generativo.

El artefacto final contiene 213,241 vectores y cubre 1,762 de 1,837 archivos pertenecientes a los tres fenomenos. Las 50 respuestas cumplen el esquema oficial: tres documentos distintos y diez fragmentos por consulta, cada texto con un maximo de 250 palabras.

Matriz de cumplimiento obligatorio

Requisito	Implementacion verificada	Estado
Encoder publico y multilingue	intfloat/multilingual-e5-base; ES/EN/PT; mismo modelo para pasajes y consultas	CUMPLE
FAISS y similitud coseno	IndexFlatIP exacto; vectores y queries L2; 213,241 filas alineadas	CUMPLE
Complejidad linguistica	Cortes en oraciones; listas, tablas y filas como unidades estructurales	CUMPLE*
Metadata obligatoria	doc_id, chunk_id, fuente, formato, fenomeno, posicion, num_tokens y texto	CUMPLE
Salida oficial	50 lineas q001-q050; 3 documentos; 10 fragmentos; maximo 250 palabras	CUMPLE
Grafo bonus	GraphML: 11,581 nodos, 16,965 aristas y evidencia doc/chunk	INCLUIDO
Persistencia y reproduccion	faiss.write_index/read_index, JSONL, generador autocontenido y Git LFS	CUMPLE

Conclusion de auditoria
La entrega obligatoria es cargable y valida. El asterisco remite a 400 eventos de omision registrados por el build; el log detallado no fue versionado y por ello no se atribuyen categorias sin evidencia.

2. Corpus, extraccion y chunking

El inventario registra 1,839 archivos: 1,837 pertenecen a F1/F2/F3 y dos son archivos de control. No hay archivos de cero bytes. Un doc_id estable se obtiene del SHA-256 de la ruta relativa, lo que mantiene la identidad entre copias del corpus sin depender de rutas absolutas.

Formato	Archivos corpus	Docs cubiertos	Chunks	Tratamiento
PDF	760	711	69,396	PyMuPDF por bloques/pagina; remueve cabeceras y pies repetidos
JSON	964	946	6,375	Campos title/body/text/...; listas de parrafos conservan orden y json_path
CSV	26	26	125,956	Una fila semantica con pares columna: valor
XLSX	6	5	1,141	Filas por hoja, cabecera como contexto
PBF	73	73	10,364	Atributos por feature y deduplicacion dentro del tile
TXT	1	1	9	Texto UTF-8 normalizado
Imagen	9	0	0	OCR opcional y trazable; no activo en este build

Cobertura observada

95.92% 1,762 / 1,837 archivos de fenomenos	75 documentos sin chunks	5,232 chunks >250 palabras indexados correctamente	478 maximo de tokens observado
--	--	---	--

Los 75 documentos no cubiertos se distribuyen en 48 PDF, 18 JSON y 9 imagenes (8 JPG, 1 AVIF). Esta cifra se calcula comparando el manifiesto con los doc_id presentes en metadata; no implica que todos sean fallas del extractor: algunos pueden carecer de texto util o contener unidades sin frontera segura.

Politica de fragmentacion

El objetivo es 360 tokens y el maximo 480. La segmentacion respeta parrafos y puntuacion terminal; los items de lista retienen sus lineas continuadas. Las filas tabulares permanecen completas salvo que una fila exceda el maximo, caso en que se agrupan campos completos separados por punto y coma. Una oracion indivisible mayor que 480 tokens se registra y se omite: nunca se corta por posicion de token.

Metrica	Tokens/chunk	Palabras/chunk
Minimo	1	1
Mediana	303	119
P95	358	239
Maximo	478	403
Promedio	298.39	136.20

Aclaracion del limite de 250 palabras

No limita el indice. Existen 5,232 chunks validos con mas de 250 palabras y hasta 478 tokens. El limite se aplica solamente a cada fragmento de resultados.jsonl, tal como exige la Seccion 9.2 del reglamento.

3. Embeddings, indice y recuperacion

Seleccion del encoder

intfloat/multilingual-e5-base ofrece un espacio comun para espanol, ingles y portugues, longitud de entrada compatible con el tope de 480 tokens y una dimension de 768 que cabe holgadamente en 8 GB de VRAM. Se anteponen los prefijos E5 exactos **passage:** y **query:**. El mismo encoder se usa en indexacion y consulta; no existe un decoder en la ruta de recuperacion.

Parametro	Valor final	Implicacion
Hardware del build	NVIDIA GeForce RTX 3050 Ti Laptop GPU	Embeddings acelerados por CUDA
Precision	float16	FP16 solo para inferencia CUDA; salida normalizada float32
Batch solicitado / efectivo	8 / 8	Sin reduccion por OOM en el build final
Tiempo registrado	64.68 s	Etapas de embeddings + escritura FAISS; no es tiempo total de extraccion
Cache	1,828 hits de extraccion; embeddings=True	Rebuild incremental sin releer/recalcular contenido intacto
Indice	IndexFlatIP; d=768	Busqueda exacta; IP equivale a coseno con L2
Grafo	11,581 nodos; 16,965 aristas	Fusion numerica no generativa; evidencia trazable
Tamanos	624.73 MiB FAISS; 333.73 MiB metadata	Artefactos versionados mediante Git LFS

Flujo de recuperacion

Consulta	E5 query + L2	FAISS top-1000	Evidencia GraphML	Top-10 / top-3
----------	---------------	----------------	-------------------	----------------

FAISS devuelve hasta 1,000 candidatos exactos. Las entidades de la consulta activan aristas del grafo y sus chunks de evidencia reciben un aporte acotado de 0.005 por evidencia (maximo 0.05); los candidatos exclusivos del grafo se incorporan al pool. Para documentos, las puntuaciones fusionadas se agrupan por doc_id mediante max pooling y se seleccionan tres ids distintos. Para fragmentos, se recorre el ranking y se divide la presentacion solo en limites linguisticos hasta completar diez textos de <=250 palabras. Si un chunk origina varios subfragmentos, estos conservan el mismo chunk_id, practica expresamente permitida por la especificacion.

Resultados y trazabilidad

Comprobacion	Resultado
Consultas y orden	50 lineas; q001 ... q050
Documentos	150 objetos; tres doc_id distintos por consulta
Fragmentos	500 objetos; diez por consulta; maximo observado = 250 palabras
Referencias	500/500 chunk_id y doc_id existen en metadata
Subfragmentacion	27 repeticiones de chunk_id dentro de consultas; permitidas por reglamento
Texto original	500/500 coincidencias literales tras normalizar espacios; 0 variaciones

Integridad vectorial
index.ntotal = metadata = 213,241; no hay chunk_id duplicados, todas las secuencias posicion empiezan en 0 y cinco vectores reconstruidos en puntos distribuidos del indice presentan norma L2 = 1.000000.

4. Reproducibilidad, validacion y riesgos

Evidencia ejecutada sobre main actualizado

Validacion	Resultado	Alcance
Git LFS	OK	Objetos de index.faiss, metadata.jsonl y encoder_config.json integros
Compilacion	OK	src, scripts, generador.py y tests
Pytest	19 passed	CUDA selection, L2, FAISS-metadata, JSON/listas/cache, esquema y grafo
Preflight	PASSED	Carga FAISS, metadata, 50 queries, 3 documentos, 10 fragmentos, <=250 palabras
Smoke CUDA	OK	Python 3.11.9; torch 2.9.1+cu128; RTX 3060 Ti; E5 + FAISS minimo
Reproduccion	Identica	50/50 rankings de documentos; 50/50 rankings de fragmentos identicos

Reproduccion verificada

La regeneracion desde la carpeta empaquetada en RTX 3060 Ti y FP16 produjo 50/50 rankings documentales y 50/50 rankings de fragmentos identicos. La identidad se garantiza para el entorno auditado; otro hardware o dtype puede alterar empates numericos.

Riesgos y limitaciones transparentes

Prioridad	Hallazgo	Impacto / tratamiento
Media	400 eventos de falla u omision en build_summary	El log detallado no esta en Git; no es posible separar bloque, OCR o fuente. Conservar extraction_failures.jsonl en la proxima entrega de auditoria.
Media	75 documentos sin chunks	4.08% de los archivos F1/F2/F3: 48 PDF, 18 JSON y 9 imagenes. Revisar PDFs escaneados/JSON sin campos TEXT_KEYS y activar OCR trazable cuando aporte semantica.
Baja	No existe ground truth publico	El peso del grafo se eligio con proxies de similitud, diversidad y trazabilidad; validar Recall@k cuando la organizacion publique juicios de relevancia.
Baja	Empates numericos entre hardware	La ejecucion auditada es identica; para auditoria inter-GPU estricta usar FP32 y fijar versiones.
Baja	Entrega separada de main	Los artefactos finales, incluido GraphML, viven en la rama entrega_final. Identificar la revision final expresamente al entregar o integrarla despues de cerrar la competencia.

Huellas SHA-256 para auditoria

```
index.faiss      27556c1fb189d9f483b673d57dc8f6e8d549bd48eb9cd6ba273c2b5b20eaaaf45
metadata.jsonl   27b809e5198c82ce6a2ddd6d6e171f3bf9d3adbbd2e37ad88690ae029ef3bc6
resultados.jsonl 2933366f4e823baab0bb3539ecb0e21f5d487002c8faa2e6dca19b4b57af4dd4
grafo.graphml    2bfe00d7de2624039f6b46c3e92b0089e5a9435afd54173c8a29fa8b03bdbb39
```

5. Arquitectura final, bonus y operacion

Separacion de responsabilidades

Componente	Responsabilidad	Garantia principal
extract.py / chunking.py	Extraccion multiformato y unidades completas	Texto trazable, maximo 480 tokens
cache.py / vector.py	Cache SHA-256/SQLite, E5, OOM fallback, L2 y FAISS	Mismo espacio semantico y rebuild incremental
retrieval.py / generador.py	Carga, busqueda, agregacion y formato oficial	Top-10 chunks, top-3 documentos, sin generacion
validate.py / validate_delivery.py	Esquema, ids, alineacion y preflight	Falla temprana ante una entrega incoherente
Git LFS	Distribucion de indice y metadata grandes	Clon reproducible sin reindexar

Grafo de conocimiento opcional

La entrega incluye un bonus determinista basado en NetworkX: detecta entidades multilingues por lexico/patrones, extrae relaciones solo cuando existe un verbo disparador y conserva doc_id/chunk_id como evidencia. El GraphML versionado contiene 11,581 nodos y 16,965 aristas (6.07 MiB). La auditoria encontro 0 referencias de chunk huérfanas, 0 de documento y 0 aristas sin trazabilidad.

Estado del bonus
grafo.graphml, graph.py y retrieval.py estan incluidos; generador.py autodetecta el grafo junto al indice. Frente al baseline vectorial coinciden 32/50 rankings documentales y 18/50 rankings de fragmentos: la fusion fue aplicada.

Reproduccion operativa en Windows

1. Crear .venv con Python 3.11 e instalar torch 2.9.1 desde cu128; luego requirements.txt.
2. Ejecutar python scripts/gpu_smoke_test.py --device cuda.
3. Construir con build_baseline.py --device cuda --batch-size 8 o cargar el indice LFS.
4. Generar con el mismo encoder y --candidates 1000.
5. Ejecutar pytest, compileall y validate_delivery.py hasta obtener PRECHECK PASSED.

Decision final

La entrega satisface las restricciones tecnicas centrales de CODEFEST: recuperacion vectorial multilingue con fusion opcional de grafo, sin modelos generativos; indice exacto y persistente, metadata alineada, fragmentos linguisticamente completos y salida oficial valida. Las mejoras mas valiosas frente a la primera iteracion son la extraccion estructurada de JSON/listas/tablas, la recuperacion de unidades largas sin imponer 250 palabras al indice, las caches por contenido y una entrega descargable mediante LFS. Las limitaciones restantes estan cuantificadas y no invalidan el preflight; deben guiar la siguiente ronda de calidad, especialmente cobertura OCR, log de fallas y determinismo FP32.

Resultado
PRECHECK PASSED | 19 tests passed | CUDA OK | 213,241 vectores alineados | 50 consultas conformes